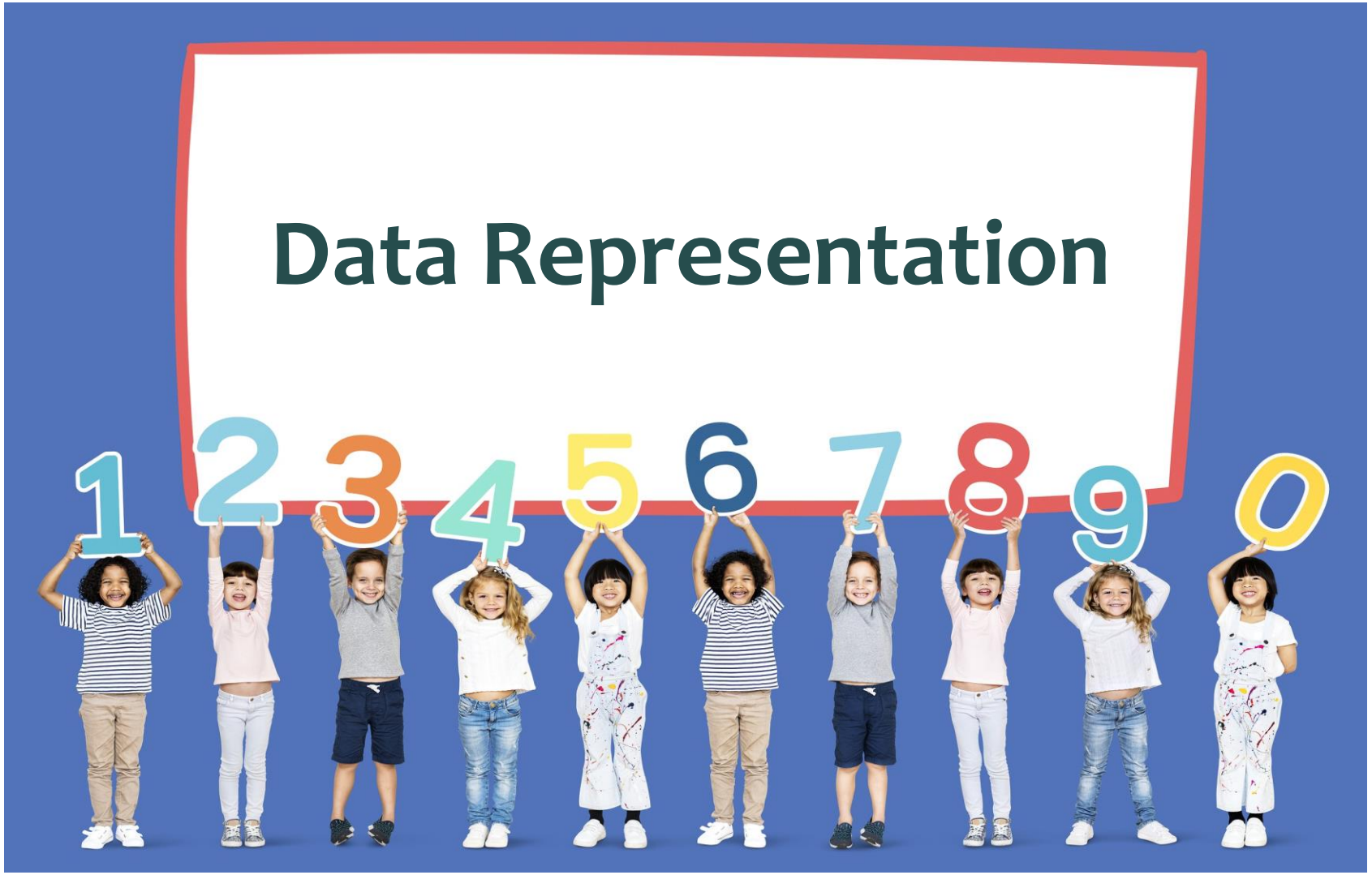
Introduction To Computer Science

Number Systems & Data Representation

Data Representation



Data Representation

Data representation can be categorized into to types

1. Qualitative
   - ASCII

2. Quantitative

2.1 Integers

- Ones Compliment
- Twos Compliment

2.2 Fractional

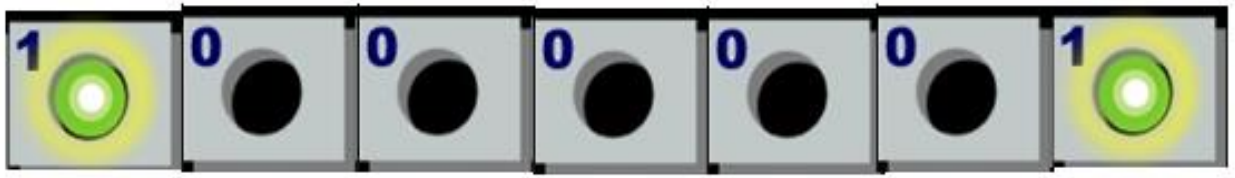
-IEEE

1. Qualitative

- ASCII - (American Standard Code for Information Interchange)

  - 7 bit code for all 128 characters

A=1000001 -> 65



1. Qualitative

ASCII Alphabet Characters

Symbol	Decimal	Binary
A	65	01000001
B	66	01000010
C	67	01000011
D	68	01000100
E	69	01000101
F	70	01000110
G	71	01000111
H	72	01001000
I	73	01001001
J	74	01001010
K	75	01001011
L	76	01001100
M	77	01001101
N	78	01001110
O	79	01001111
P	80	01010000
Q	81	01010001
R	82	01010010
S	83	01010011
T	84	01010100
U	85	01010101
V	86	01010110
W	87	01010111
X	88	01011000
Y	89	01011001
Z	90	01011010

Symbol	Decimal	Binary
a	97	01100001
b	98	01100010
c	99	01100011
d	100	01100100
e	101	01100101
f	102	01100110
g	103	01100111
h	104	01101000
i	105	01101001
j	106	01101010
k	107	01101011
l	108	01101100
m	109	01101101
n	110	01101110
o	111	01101111
p	112	01110000
q	113	01110001
r	114	01110010
s	115	01110011
t	116	01110100
u	117	01110101
v	118	01110110
w	119	01110111
x	120	01111000
y	121	01111001
z	122	01111010

ASCII Hexadecimal code

		000	001	010	011	100	101	110	111
		0	1	2	3	4	5	6	7
0000	0	NULL	DLE		0	@	P	`	p
0001	1	SOH	DC1	!	1	A	Q	a	q
0010	2	STX	DC2	"	2	B	R	b	r
0011	3	ETX	DC3	#	3	C	S	c	s
0100	4	EDT	DC4	$	4	D	T	d	t
0101	5	ENQ	NAK	%	5	E	U	e	u
0110	6	ACK	SYN	&	6	F	V	f	v
0111	7	BEL	ETB	'	7	G	W	g	w
1000	8	BS	CAN	(	8	H	X	h	x
1001	9	HT	EM	)	9	I	Y	i	y
1010	A	LF	SUB	*	:	J	Z	j	z
1011	B	VT	ESC	+	;	K	[	k	{
1100	C	FF	FS	,	<	L	\	l	|
1101	D	CR	GS	-	=	M	]	m	}
1110	E	SO	RS	.	>	N	^	n	~
1111	F	SI	US	/	?	O	_	o	DEL

Identify the message written using ASCII Hexadecimal code

1. 49 20 4C 6F 76 65 20 49 54

2. 48 65 79 21 49 20 66 6F 75 6E 64 20 69 74 2E

3. 44 61 6E 67 65 72 6F 75 73 21

4. 49 20 4C 6F 76 65 20 49 43 53

Identify the message written using ASCII Hexadecimal code

1. I Love my life!
2. Happy Halloween!
3. Good Morning!
4. See you…

1. Qualitative

- Complements : used in digital computers for simplifying
  - the subtraction operation
  - the logical manipulation
- There are two types of compliments for each base ***r*** system
  - ***r's*** compliments
  - ***(r-1)'s*** compliments
- the value r is submitted in the name,
  - 2's and 1's compliments – binary numbers
  - 10's and 9's compliments – decimal numbers

Representation of Negative Numbers

-20 = ?

- **Sign and Magnitude representation**
- **1's Complement representation**
- **2's Complement representation**

Sign and magnitude representation

In this method the binary equivalent is stored together with a sign bit given as the MSB, which is 1 if the number is negative and 0 if the number is positive.

Example: (use 8bit format)

-70 -> **1** 1000110

+20 -> **0** 0010100

Example2:

Add -70 +20 using S/M (use 8bit format)

-70 -> **1** 1000110

+20 -> **0** 0010100

---

1 1011010

(Ans. Is wrong, since its -90. It should be -50)

1's Complement representation

The positive numbers are represented in similar way however inverting all the bits represents the negative numbers and there is no separate sign bit.

Example: (use 8bit format)

70	->	01000110	
- 70	->	10111001	(-70 in 1's Compt.)

Example2:

Add -70 +20 using 1's Complement (use 8bit format)

-70	->	10111001	(-70 in 1's Compt.)
+20	->	00010100	

		11001101	(-50 in 1's Compt.)

(If 11001101 is -50, 00110010 Is 50)

Example3:

Add -48 +72 using 1's Complement (use 8bit format)

72	->	01001000	
48	->	00110000	
- 48	->	11001111	(-48 in 1's Compt.)

72	->	01001000	
- 48	->	11001111	(-48 in 1's Compt.)

```
             -------------
             100010111
             |──────────> 1
             --------------
             00011000      (Is 24 in Decimal)
```

2's complement representation

Same as above but you need to add 1 one after inverting all the bits. Both these allow you to easily perform subtraction but 2's complement is better because the final carry can be ignored.

Example: (use 8bit format)

```
 70        ->    01000110
-70        ->    10111001
                 +       1
                 ----------
                 10111010      (-70 in 2's Compt.)
```

Example2:

Add -70 +20 using 2's Complement (use 8bit format)

```
-70    ->    10111010    (-70 in 2's Compt.)
+20    ->    00010100
             ----------
             11001110    (-50 in 2's Compt.)
```

```
11001110 (-50 in 2's Compt.)
       1
---------------
11001101
00110010 (+50 in 2's Compt.)
```

Example3:

Add 90 + (-52) using 2’s Complement (use 8bit format)

```
90          ->      01011010
                                +
52          ->      00110100
                    11001011
                           1
                    -------------
-52     ->          11001100      (-52 in 2’s Compt.)
```

```
 90     ->      01011010
-52     ->      11001100      (-52 in 2’s Compt.)
                -------------
                 100100110   (Is 38 in Decimal)
```

Perform the following arithmetic operations using two's complement and comment on the results (use 8 bit numbers).

a) +39 - 75
b) -20 - 110

```
a)      75      ->      01001011
                ->      10110100
                                1
                        ---------
        -75             10110101      (-75 in 2's Compt.)
        39              00100111
                        ---------
                        11011100      (-36 in 2's Compt.)
```

```
11011100   (-36 in 2's Compt.)
       1
----------
11011011
00100100   (+36 in Decimal)
```

b) -20 - 110

b)

```
110     ->      01101110
        ->      10010001
                       1  +
                -------------
-110            10010010      (-110 in 2's Compt.)

20      ->      00010100
        ->      11101011
                       1  +
                -------------
        -20             11101100      (-20 in 2's Compt.)

-110    ->      10010010      (-110 in 2's Compt.)
-20     ->      11101100  +   (-20 in 2's Compt.)
                -------------
                 101111110    (-130 in 2's Compt.)
                 x
```

```
01111110  (-130 in 2's Compt.)
       1
-------------
01111101
10000010 (130 in Decimal)
```

Zero “0” represented in 1’s and 2’s Complement:

1’s Complement:

0 -> 00000000
-0 -> 11111111

2’s Complement:

0 -> 00000000

-0 -> 00000000
11111111
1

100000000
X

2. Quantitative

- Fractional

  - IEEE 754

1. Convert Floating point number into Binary

   ex : 5.375

	375 x 2
0	750 x 2
1	500 x 2
1	000

101.011

2. Quantitative

- Fractional

  - IEEE 754

1. Convert Binary to Decimal

ex : 1 0 1 . 1 0 1

1	**0**	**1**	**.**	**1**	**0**	**1**
2^2	2^1	2^0		2^{-1}	2^{-2}	2^{-3}
4	2	1		1/2	1/4	1/8
4+	0 +	1+		½ +	¼ +	1/8

$\boxed{5.625}$

Convert

To Binary	To Decimal
1. 2.75	1. 111.11
2. 45.5	2.1010.1
3. 5.125	3. 110.101
4. 11.34	4. 1101.01
5. 126.3	5. 11010.011

2. Quantitative

- Fractional

  - IEEE 754

2. Scientific Notation

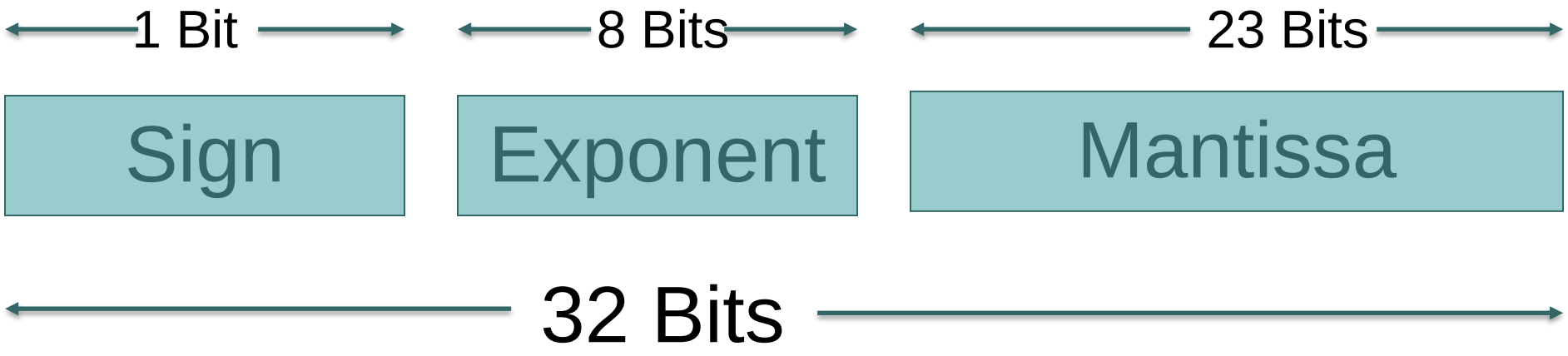
ex : 1. 1 0 1 . 1 0 1 ⇒ 1.01101×2^2

2. 1 0 0 1 . 0 1 0 1 1.0010101×2^3

2. Quantitative

- Fractional

  - IEEE 754 – Single Precision



2. Quantitative

- Fractional

  - IEEE 754 – Single Precision

Ex : 1.01101×2^{2}

Mantissa

Exponent

2. Quantitative

- Fractional

  - IEEE 754 – Single Precision

Ex : 1.01101×2^2

$=2^{(n-1)} - 1$

Sign	Exponent	Mantissa
0	10000001	01101000000000000000000

0 if positive
1 if Negative

E = 127 + 2

M has 23 bits

2. Quantitative

- Fractional

  - IEEE 754 – Single Precision

- Ex : $0.250_{10} \longrightarrow 0.01_2 \longrightarrow 1.0 \times 10^{-2}$

Sign	Exponent	Mantissa
0	1111101	00000000000000000000000

0 if positive
1 if Negative

E = 127 + (-2)

M has 23 bits

Exercise

The IEEE 754 standard for binary floating point arithmetic defines what is commonly referred to as "IEEE floating point".

```
Example: 85.125
   1.   Convert to Base2
        85 = 1010101
        0.125 = 001
        85.125 = 1010101.001

   2.   Normalize
        1010101.001
       =1.010101001 x 2^6

   3.   Add 127 to exponent field and convert to binary:
        exponent 127+6=133
        133 = 10000101

   4.   Determine the sign bit. If a negative, set to 1. Else set to 0.
        sign = 0

   5.   Now put the numbers together:
        we will add 0's to complete the 23 bits

        0 10000101 01010100100000000000000
```

Exercise

1. 19.375 ->
2. 0.9375 ->
3. -123.3 ->

Exercise

1. 19.375 ->

0	**10000011**	**00110110000000000000000**

1. 0.9375 ->

0	**01111101**	**11100000000000000000000**

1. -123.3 ->

1	**10000101**	**11100101001101001100100**